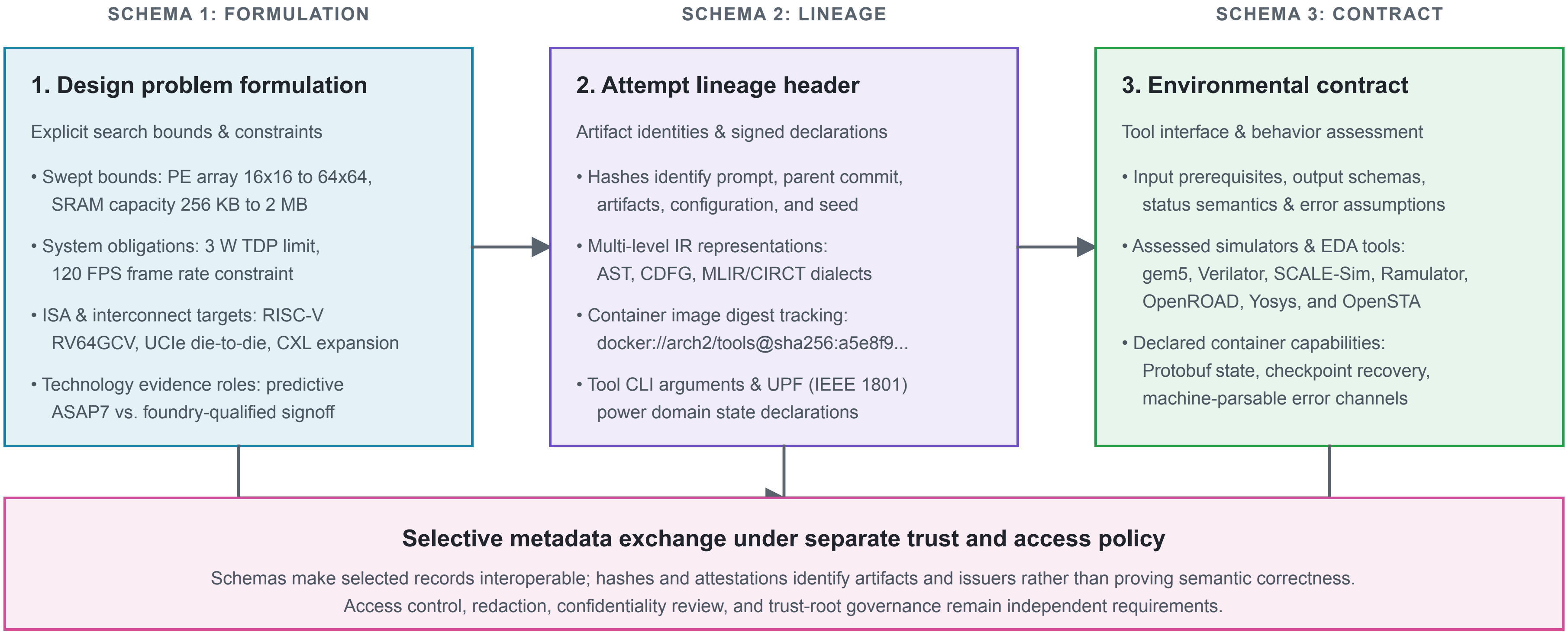
SCHEMA 1: FORMULATION
SCHEMA 2: LINEAGE
SCHEMA 3: CONTRACT
1. Design problem formulation
Explicit search bounds & constraints
• Swept bounds: PE array 16x16 to 64x64, SRAM capacity 256 KB to 2 MB
• System obligations: 3 W TDP limit, 120 FPS frame rate constraint
• ISA & interconnect targets: RISC-V RV64GCV, UCIe die-to-die, CXL expansion
• Technology evidence roles: predictive ASAP7 vs. foundry-qualified signoff
2. Attempt lineage header
Artifact identities & signed declarations
• Hashes identify prompt, parent commit, artifacts, configuration, and seed
• Multi-level IR representations: AST, CDFG, MLIR/CIRCT dialects
• Container image digest tracking: docker://arch2/tools@sha256:a5e8f9...
• Tool CLI arguments & UPF (IEEE 1801) power domain state declarations
3. Environmental contract
Tool interface & behavior assessment
• Input prerequisites, output schemas, status semantics & error assumptions
• Assessed simulators & EDA tools: gem5, Verilator, SCALE-Sim, Ramulator, OpenROAD, Yosys, and OpenSTA
• Declared container capabilities: Protobuf state, checkpoint recovery, machine-parsable error channels
Selective metadata exchange under separate trust and access policy
Schemas make selected records interoperable; hashes and attestations identify artifacts and issuers rather than proving semantic correctness.
Access control, redaction, confidentiality review, and trust-root governance remain independent requirements.